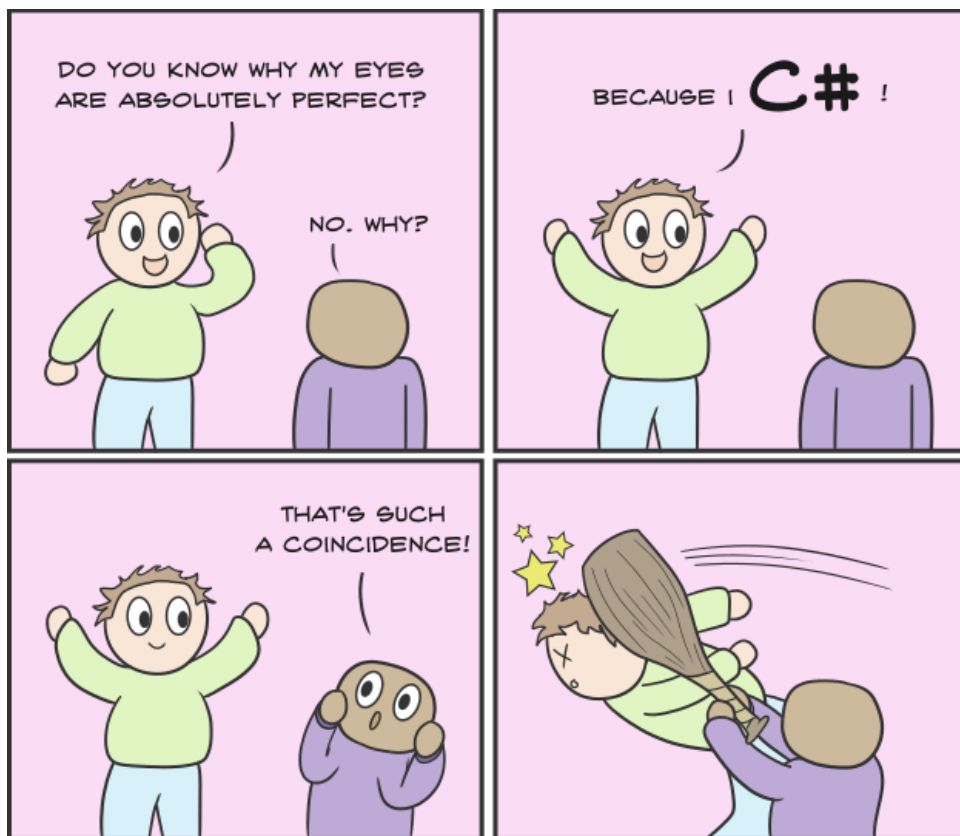


EL Manual DORADO de C#

Introdução à Programação com Visual Studio Community

“Há quem diga que programar é fácil e divertido de aprender” - Também há quem se engane



Módulo 1 O que é Programação e o C#?

O que é um programa?

Um programa é um conjunto de instruções que dizemos ao computador para ele executar. O computador é muito rápido, mas não sabe pensar sozinho — precisamos de dizer-lhe exatamente o que fazer, passo a passo.

Imagina que estás a dar indicações a alguém que nunca saiu de casa. Tens de ser muito preciso: **"Vira à esquerda, anda 50 passos, abre a porta verde..."** — a programação é exatamente isso!

O que é o C#?

C# (lê-se "C Sharp") é uma linguagem de programação criada pela Microsoft. É usada para criar:

- Aplicações para Windows
- Jogos (com Unity)
- Sites e APIs
- Aplicações móveis

É uma linguagem moderna, segura e muito popular no mercado de trabalho!

O Visual Studio Community

O Visual Studio Community é o programa onde vamos escrever o nosso código C#. É gratuito e é o mesmo ambiente que os programadores profissionais usam.

Criar o Primeiro Projeto — Passo a Passo

1. Abre o Visual Studio Community
2. Clica em «Criar um novo projeto»
3. Escolhe «Aplicação de Consola» (Console App)
4. Clica em «Seguinte»
5. Dá um nome ao projeto (ex: MeuPrimeiroProjeto)
6. Clica em «Seguinte»

ATENÇÃO: Na próxima janela vais ver uma opção:

«Do not use top-level statements»

Garante que esta opção está MARCADA!

Sem ela, o C# esconde a estrutura real do programa.

7. Clica em «Criar»

Depois de criar o projeto, vais ver este código automaticamente:

```
namespace MeuPrimeiroProjeto
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

O que é cada parte?

namespace -> Agrupa o código (como uma pasta)

class Program -> O nosso 'ficheiro' de código

static void Main -> O ponto de partida do programa (começa SEMPRE aqui)

Console.WriteLine() -> Mostra texto no ecrã

Exercício 1.1

Cria o teu primeiro projeto no Visual Studio e corre-o (botão verde).

1. O programa mostra 'Hello, World!' no ecrã?
2. Altera o texto para 'Olá, Mundo!' e corre novamente.
3. Adiciona uma segunda linha: `Console.WriteLine("Bem-vindo ao C#!");`

Módulo 2 Variáveis e Tipos de Dados

O que é uma variável?

Uma variável é como uma caixa com um nome onde guardamos um valor. Podemos guardar números, texto, verdadeiro/falso, etc.

Pensa num cacifo da escola: tem um número (nome), e podes guardar coisas lá dentro e trocar quando quiseres.

Tipos de variáveis mais usados

Tipo	Para que serve	Exemplo
int	Números inteiros	<code>int idade = 17;</code>
double	Números com vírgula	<code>double nota = 13.5;</code>
string	Texto	<code>string nome = "Ana";</code>
bool	Verdadeiro ou Falso	<code>bool passou = true;</code>
char	Um único caractere	<code>char letra = 'A';</code>

e float ? Como será que eu podia declarar e inicializar um float?

E qual é a diferença entre um float e um double?



Como declarar e usar variáveis

```
static void Main(string[] args)
{
    // Declarar e atribuir ao mesmo tempo
    int idade = 16;
    string nome = "Maria";
    double altura = 1.65;
    bool alunaEsforcada = true;

    // Mostrar no ecrã
    Console.WriteLine(nome);
    Console.WriteLine(idade);

    // Juntar texto com variáveis
    Console.WriteLine("Olá, " + nome + "! Tens " + idade + " anos.");

    // Forma moderna (interpolação de strings)
    Console.WriteLine($"Olá, {nome}! Tens {idade} anos.");
}
```

Operadores Aritméticos

```
int a = 10;
int b = 3;

Console.WriteLine(a + b);    // Soma      -> 13
Console.WriteLine(a - b);    // Subtração -> 7
Console.WriteLine(a * b);    // Multiplic. -> 30
Console.WriteLine(a / b);    // Divisão   -> ?? O que será que sai daqui?
Console.WriteLine(a % b);    // Resto     -> 1

double x = 10.0;
double y = 3.0;
Console.WriteLine(x / y);    // E daqui? O que será que sai?
```

Exercício 2.1

Cria um programa com variáveis sobre ti próprio.

1. Cria variáveis: nome (string), idade (int), altura (double), estudante (bool).
2. Mostra uma frase usando todas as variáveis. Ex: "O João tem 17 anos, mede 1.75m."
3. Calcula a soma, diferença e produto de dois números à tua escolha e mostra os resultados.

4. Declara uma variável 'anoNascimento', calcula o ano atual menos idade e mostra o ano em que nasceste!

Módulo 3 Ler Dados do Utilizador

Console.ReadLine()

Até agora o programa só fala. Agora vai ouvir! O Console.ReadLine() pausa o programa e espera que o utilizador escreva algo e carregue Enter.

Atenção: o ReadLine() devolve sempre uma string (texto). Se precisarmos de um número, temos de converter!

Exemplo — Nome do utilizador

```
static void Main(string[] args)
{
    Console.Write("Como te chamas? ");
    string nome = Console.ReadLine();
    Console.WriteLine($"Olá, {nome}! Bem-vindo ao C#!");
}
```

Write vs WriteLine

Console.Write() -> Escreve sem saltar linha

Console.WriteLine() -> Escreve E salta para a linha seguinte

Converter texto em número

```
Console.Write("Qual é a tua idade? ");
string textoIdade = Console.ReadLine();
int idade = int.Parse(textoIdade); // Converte string -> int

// Ou numa só linha:
int idade2 = int.Parse(Console.ReadLine());

// Para double:
double nota = double.Parse(Console.ReadLine());

Console.WriteLine($"Daqui a 5 anos terás {idade + 5} anos.");
```

Exemplo completo — Calculadora simples

```
static void Main(string[] args)
{
    Console.WriteLine("=== Calculadora ===");

    Console.Write("Primeiro número: ");
    double num1 = double.Parse(Console.ReadLine());

    Console.Write("Segundo número: ");
    double num2 = double.Parse(Console.ReadLine());

    Console.WriteLine($"Soma:          {num1 + num2}");
    Console.WriteLine($"Diferença:   {num1 - num2}");
    Console.WriteLine($"Produto:    {num1 * num2}");
    Console.WriteLine($"Quociente:  {num1 / num2}");
}
```

Exercício 3.1

Programa interativo com o utilizador.

1. Pede o nome e a idade ao utilizador e mostra uma saudação personalizada.
2. Pede dois números e mostra a soma, diferença, produto e divisão.
3. Pede o nome de um produto e o preço. Calcula o preço com 23% de IVA e mostra o resultado.
4. Pede a nota de 3 testes (0-20) e calcula a média. Mostra: "A tua média é X."

Módulo 4 Condições — if, else if, else

O que são condições?

Um programa muitas vezes precisa de tomar decisões: "SE isto for verdade, faz X; SENÃO faz Y." É exatamente isso que o if/else faz!

Pensa numa portagem: SE a via está verde, passa; SENÃO, para e paga.

Operadores de comparação

==	igual a	(5 == 5 -> true)
!=	diferente de	(5 != 3 -> true)
>	maior que	(7 > 4 -> true)
<	menor que	(3 < 9 -> true)
>=	maior ou igual que	(5 >= 5 -> true)
<=	menor ou igual que	(4 <= 6 -> true)

Operadores lógicos

&&	E (AND) — ambas têm de ser verdade
	OU (OR) — basta uma ser verdade
!	NÃO (NOT) — inverte o resultado

IF simples

Executa o bloco APENAS se a condição for verdadeira. Se for falsa, não faz nada.

<code>int temperatura = 35;</code>
<code>if (temperatura > 30)</code>
<code>{</code>
<code> Console.WriteLine("Está muito calor!");</code>
<code>}</code>
<code>Console.WriteLine("Programa continua...");</code>

Lê assim:

SE (temperatura > 30) ENTÃO mostra 'Está muito calor!'

IF / ELSE

Executa um bloco SE a condição for verdadeira, e OUTRO bloco se for falsa. Um dos dois SEMPRE executa.

```
int nota = 9;

if (nota >= 10)
{
    Console.WriteLine("Aprovado! Parabéns!");
}
else
{
    Console.WriteLine("Reprovado. Tenta outra vez!");
}
```

Lê assim:

SE (nota >= 10) ENTÃO mostra 'Aprovado'
SENÃO mostra 'Reprovado'

Outro exemplo — verificar se um número é par ou ímpar:

```
Console.Write("Escreve um número: ");
int numero = int.Parse(Console.ReadLine());

if (numero % 2 == 0)
{
    Console.WriteLine($"{numero} é par.");
}
else
{
    Console.WriteLine($"{numero} é ímpar.");
}
```

IF / ELSE IF / ELSE

Quando temos mais de duas opções! O C# verifica cada condição em ordem — assim que encontra uma verdadeira, executa e sai. O else final apanha tudo o que não foi apanhado antes.

```
int nota = 14;

if (nota >= 18)
{
    Console.WriteLine("Excelente! Muito Bom!");
}
else if (nota >= 14)
{
    Console.WriteLine("Bom!");
}
else if (nota >= 10)
{
    Console.WriteLine("Suficiente. Aprovado.");
}
else
{
    Console.WriteLine("Insuficiente. Reprovado.");
}
```

Lê assim:

SE nota >= 18 -> Excelente
SENÃO SE nota >= 14 -> Bom
SENÃO SE nota >= 10 -> Suficiente
SENÃO -> Insuficiente

Exemplo — classificar a temperatura:

```

Console.Write("Qual é a temperatura? ");
double temp = double.Parse(Console.ReadLine());

if (temp < 0)
{
    Console.WriteLine("Congelante! Fica em casa.");
}
else if (temp < 10)
{
    Console.WriteLine("Muito frio. Casaco obrigatório!");
}
else if (temp < 20)
{
    Console.WriteLine("Fresco. Leva um casaco.");
}
else if (temp < 30)
{
    Console.WriteLine("Agradável. Boa temperatura!");
}
else
{
    Console.WriteLine("Calor intenso. Hidrata-te!");
}

```

IF / IF (condições independentes)

Às vezes queremos verificar várias condições que NÃO estão relacionadas entre si. Usamos IFs separados — cada um é verificado sempre, independentemente dos outros.

```

int pontos = 85;
bool presencaOk = true;

// Estes IFs são INDEPENDENTES — ambos são sempre verificados
if (pontos >= 50)
{
    Console.WriteLine("Passou pela nota!");
}

if (presencaOk == true)
{
    Console.WriteLine("Presença em dia!");
}

```

IF vs ELSE IF — Qual usar?

IF separados -> Condições INDEPENDENTES (verifica sempre todas)
 ELSE IF -> Condições ALTERNATIVAS (mal encontra uma, para)

Exemplo: verificar nota E presença -> IFs separados
Exemplo: classificar nota (Excelente/Bom/Suficiente) -> ELSE IF

IF com operadores lógicos (&& e ||)

Podemos combinar condições numa só com && (E) e || (OU):

```
int nota = 12;
int faltas = 3;

// && - ambas têm de ser verdade
if (nota >= 10 && faltas <= 5)
{
    Console.WriteLine("Aprovado com presença!");
}

// || - basta uma ser verdade
if (nota >= 18 || faltas == 0)
{
    Console.WriteLine("Aluno de destaque!");
}

// ! - negação
bool temManuais = false;
if (!temManuais)
{
    Console.WriteLine("Não tens os manuais!");
}
```

IF dentro de IF (encadeados)

Podemos colocar um IF dentro de outro IF. Útil quando a segunda condição só faz sentido se a primeira for verdadeira:

```
Console.Write("Tens cartão de sócio? (sim/nao): ");
string cartao = Console.ReadLine();

if (cartao == "sim")
{
    Console.Write("Qual é a tua idade? ");
    int idade = int.Parse(Console.ReadLine());

    if (idade < 12)
    {
        Console.WriteLine("Entrada grátis!");
    }
    else
```

```
{  
    Console.WriteLine("Desconto de 20%!");  
}  
  
}  
  
else  
{  
    Console.WriteLine("Preço normal.");  
}
```

Exercício 4.1

Exercícios de if / else / else if

1. Pede uma nota (0-20). Mostra: Excelente (≥ 18), Bom (≥ 14), Suficiente (≥ 10) ou Insuficiente.
2. Pede dois números e diz qual é o maior, ou se são iguais.
3. Pede o salário de alguém. Se > 2000 -> imposto de 28%. Se > 1000 -> 15%. Senão -> 0%. Mostra o salário líquido.
4. Pede a hora (0-23). Mostra: 'Bom dia' (6-12), 'Boa tarde' (12-19), 'Boa noite' (restante).
5. DESAFIO: Pede ano de nascimento. Calcula a idade. Se ≥ 18 mostra 'Maior de idade', senão mostra quantos anos faltam.

Módulo 5 Ciclos — for, while, do...while

O que são ciclos?

Imagina que tens de escrever 100 vezes no quadro 'Não vou falar na aula'. Sem ciclos, terias de escrever 100 `Console.WriteLine(...)`. Com ciclos, escrevemos uma vez e dizemos quantas vezes repetir!

Os ciclos permitem repetir um bloco de código enquanto uma condição for verdadeira.

Ciclo FOR

O for é ideal quando sabemos EXATAMENTE quantas vezes queremos repetir.

```
// for ( início ; condição ; incremento )
```

```
for (int i = 1; i <= 5; i++)
```

```
{
```

```
    Console.WriteLine($"Contagem: {i}");
```

```
}
```

```
// Resultado:
```

```
// Contagem: 1
```

```
// Contagem: 2
```

```
// Contagem: 3
```

```
// Contagem: 4
```

```
// Contagem: 5
```

Como funciona o FOR?

`int i = 1` -> começa com `i = 1`

`i <= 5` -> continua enquanto `i` for menor ou igual a 5

`i++` -> no fim de cada volta, `i` aumenta 1 (`i++` é o mesmo que `i = i + 1`)

Contar ao contrário:

```
for (int i = 10; i >= 1; i--)
```

```
{
```

```
    Console.WriteLine($"Contagem decrescente: {i}");
```

```
}
```

```
Console.WriteLine("BOOM!");
```

Tabuada do 5 com for:

```
int numero = 5;
```

```
for (int i = 1; i <= 10; i++)
```

```
{
```

```
    Console.WriteLine($"{numero} x {i} = {numero * i}");
```

```
}
```

Soma de números com for:

```
int soma = 0;

for (int i = 1; i <= 100; i++)
{
    soma = soma + i;    // ou: soma += i;
}

Console.WriteLine($"A soma de 1 a 100 é: {soma}");
// Resultado: 5050
```

Exercício 5.1

Exercícios com FOR

1. Mostra os números de 1 a 20.
2. Mostra apenas os números PARES de 2 a 20.
3. Pede um número ao utilizador e mostra a sua tabuada completa (x1 a x10).
4. Soma todos os números de 1 a N (pede N ao utilizador).
5. DESAFIO: Mostra os primeiros 10 múltiplos de um número pedido ao utilizador.

Ciclo WHILE

O while é ideal quando NÃO sabemos quantas vezes vai repetir — depende de uma condição que pode mudar. Repete enquanto a condição for verdadeira.

```
int numero = 1;

while (numero <= 5)
{
    Console.WriteLine($"Número: {numero}");
    numero++;    // IMPORTANTE: sem isto, ciclo infinito!
}
```

Cuidado com ciclo infinito!

Se a condição nunca ficar falsa, o programa fica preso para sempre!

Garante sempre que algo muda dentro do while para que a condição eventualmente seja falsa.

Exemplo real — adivinhar um número:

```
int segredo = 7;
int tentativa = 0;

while (tentativa != segredo)
{
    Console.Write("Adivinha o número (1-10): ");
```



```
tentativa = int.Parse(Console.ReadLine());

if (tentativa < segredo)
    Console.WriteLine("Muito baixo!");
else if (tentativa > segredo)
    Console.WriteLine("Muito alto!");
}

Console.WriteLine("Parabéns! Acertaste!");
```

Exemplo — menu que repete até o utilizador sair:

```
string opcao = "";

while (opcao != "0")
{
    Console.WriteLine("=== MENU ===");
    Console.WriteLine("1 - Dizer olá");
    Console.WriteLine("0 - Sair");
    Console.Write("Escolha: ");
    opcao = Console.ReadLine();

    if (opcao == "1")
        Console.WriteLine("Olá!");
    else if (opcao != "0")
        Console.WriteLine("Opção inválida.");
}

Console.WriteLine("Até logo!");
```

Exercício 5.2

Exercícios com WHILE

1. Mostra os números de 1 a 10 com while.
2. Pede números ao utilizador até ele escrever 0. No final mostra a soma de todos.
3. Jogo: gera um número secreto (ex: 42). Pede ao utilizador para adivinhar e dá dicas (alto/baixo) até acertar.
4. Cria um programa que pede uma senha. Se a senha for errada, pede novamente. Máximo 3 tentativas.

Ciclo DO...WHILE

O do...while é parecido com o while, mas com uma diferença CRUCIAL: o código dentro do do executa SEMPRE pelo menos uma vez, e SÓ DEPOIS verifica a condição.

```
// WHILE — pode não executar nenhuma vez

int x = 100;

while (x < 10)
```

```

{
    Console.WriteLine("Isto nunca aparece!");
}

// DO WHILE — executa SEMPRE pelo menos uma vez
do
{
    Console.WriteLine("Isto aparece uma vez mesmo que a condição seja falsa!");
} while (x < 10); // condição verificada NO FIM

```

Uso clássico — validar input do utilizador:

```

int nota;

do
{
    Console.Write("Insere uma nota entre 0 e 20: ");
    nota = int.Parse(Console.ReadLine());

    if (nota < 0 || nota > 20)
        Console.WriteLine("Nota inválida! Tenta outra vez.");
} while (nota < 0 || nota > 20);

Console.WriteLine($"Nota registada: {nota}");

```

Quando usar cada ciclo?

FOR -> Sabes exatamente quantas vezes vai repetir
WHILE -> Não sabes quantas vezes — depende de uma condição
DO WHILE -> Igual ao while, mas tem de executar PELO MENOS uma vez

Comparação lado a lado:

```

// FOR — 5 vezes
for (int i = 0; i < 5; i++)
    Console.WriteLine($"for: {i}");

// WHILE — 5 vezes
int j = 0;
while (j < 5)
{
    Console.WriteLine($"while: {j}");
    j++;
}

// DO WHILE — 5 vezes

```

```
int k = 0;
do
{
    Console.WriteLine($"do while: {k}");
    k++;
} while (k < 5);
```

Exercício 5.3

Exercícios com DO...WHILE

1. Pede ao utilizador que adivinhe um número entre 1 e 5 até acertar (usa do...while).
2. Mostra um menu com pelo menos 3 opções e repete até o utilizador escolher 'Sair'.
3. Pede uma senha ao utilizador. Usa do...while para repetir até a senha ser 'abc123'.

Módulo Final Projeto Final — Junta Tudo!

Agora que já sabes variáveis, input, condições e ciclos, estás pronto para criar um programa completo!

Exercícios de revisão

Exercício Final 1 — Ficha de Aluno

Cria um programa que:

1. Pede:
 - Nome
 - Idade
 - Curso
 - Nota final
2. Mostra uma ficha organizada:

=== FICHA DO ALUNO ===

Nome: João

Idade: 17

Curso: Programação

Nota Final: 15

3. Usa `if/else` para mostrar:
 - “Aprovado” se nota ≥ 10
 - “Reprovado” caso contrário
 4. Mostra também:
 - “Excelente aluno!” se nota ≥ 18
-

Exercício Final 2 — Mini Calculadora

Cria uma calculadora que:

1. Pede dois números
2. Mostra:
 - Soma
 - Subtração
 - Multiplicação
 - Divisão
3. Usa `if` para impedir divisão por zero.

Exemplo:

Não é possível dividir por zero!

Exercício Final 3 — Tabuada Completa

Pede um número ao utilizador e mostra:

- A tabuada do 1 ao 10
- Apenas com `for`

Exemplo

7 x 1 = 7

7 x 2 = 14

...

Exercício Final 4 — Verificador de Idade

Cria um programa que:

1. Pede a idade
2. Mostra:
 - Criança (<13)
 - Adolescente (13-17)
 - Adulto (18-64)
 - Sénior (65+)

Usa `if` / `else if` / `else`.

Exercício Final 5 — Média de Notas

Cria um programa que:

1. Pede 3 notas
 2. Calcula a média
 3. Mostra:
 - “Excelente” se média ≥ 18
 - “Bom” se média ≥ 14
 - “Suficiente” se média ≥ 10
 - “Insuficiente” caso contrário
-

Exercício Final 6 — Soma Até Zero

Cria um programa com `while` que:

1. Vai pedindo números ao utilizador
2. Continua até o utilizador escrever 0
3. No final mostra:

- Soma total
 - Quantos números foram inseridos
-

Exercício Final 7 — Login Simples

Cria um sistema de login:

- Username correto: `admin`
- Password correta: `1234`

O utilizador tem:

- Máximo 3 tentativas

Mostra:

Login efetuado com sucesso!

ou

Conta bloqueada.

Exercício Final 8 — Menu Interativo

Cria um menu usando `do...while`:

- 1 - Dizer Olá
- 2 - Mostrar Hora
- 3 - Mostrar Nome
- 0 - Sair

O menu deve repetir até o utilizador escolher `0`.

Exercício Final 9 — Jogo de Adivinhação

Cria um jogo onde:

1. O programa define um número secreto
2. O utilizador tenta adivinhar
3. O programa responde:
 - “Muito alto”
 - “Muito baixo”
 - “Acertaste!”

Usa `while`.

DESAFIO:

Conta quantas tentativas o utilizador precisou.

Exercício Final 10 — Caixa de Supermercado

Cria um programa que:

1. Pede:
 - Nome do produto
 - Preço
 - Quantidade
2. Calcula:
 - Total sem IVA
 - IVA a 23%
 - Total final
3. Se o total for superior a 100€:
 - Dá desconto de 10%

Mostra tudo organizado no ecrã.

Exercício Final 11 — Conversor de Temperaturas

Cria um programa que:

1. Pede uma temperatura em Celsius
2. Converte para Fahrenheit

Fórmula:

$$F = (C * 9 / 5) + 32$$

Depois mostra:

- “Muito frio”
- “Agradável”
- “Muito quente”

Dependendo do valor.

Exercício Final 12 — Sistema de Multas

Cria um programa que:

1. Pede a velocidade do carro
2. Se:
 - $\leq 50 \rightarrow$ “Velocidade permitida”
 - 51–80 $\rightarrow$ “Multa leve”
 - 80 $\rightarrow$ “Multa grave”

DESAFIO:

Calcula um valor de multa diferente para cada caso.

Desafio 13 — Número Primo

Pede um número e verifica se é primo.

Exemplo:

7 -> Primo

8 -> Não primo

Desafio 14 — Mini Banco

Cria um sistema simples:

- Saldo inicial: 1000€
- Menu:
 - Ver saldo
 - Depositar
 - Levantar
 - Sair

Não permitir levantar mais dinheiro do que existe.

Desafio 15 — Contagem Regressiva

Pede um número e faz contagem regressiva até 0.

Exemplo:

10

9

8

...

0

BOOM!

Desafio 16 — Pedra, Papel ou Tesoura

Cria um jogo simples:

- Jogador escolhe:
 - pedra
 - papel
 - tesoura
- O computador escolhe aleatoriamente
- O programa diz quem ganhou

DESAFIO EXTRA OPCIONAL:

Adicionar pontuação até alguém ganhar 3 vezes.

Boa sorte e bom código!